Collatz Parity Encoding: A Reversible, Metadata-Free Scheme Using the Collatz Conjecture and Binary Delimiters

June 2, 2025

Abstract

We introduce the Collatz Parity Encoding (CPE), a novel reversible encoding scheme based on the parity sequences generated by the Collatz Conjecture. This method maps characters to binary parity sequences derived from the trajectory of numbers under the Collatz function. However, the parity sequences are not prefix free, necessitating the use of an explicit delimiter to separate encoded characters. Consequently, the current implementation does not achieve fully self-delimiting behavior. This paper details the encoding and decoding algorithms, analyzes the prefix properties of the sequences, and discusses the potential of parity-based encodings for compact and reversible data representation with future work aimed at developing delimiter-free decoding methods.

Keywords: Collatz Conjecture, Parity Sequence, Unique Delimiters, Data Encoding, Potential Prefix-Free Code, Information Theory

1 Introduction

Data encoding schemes are essential in computing and communication, especially when metadata such as lengths or delimiters cannot be used. Inspired by the Collatz Conjecture, we explore a novel approach to encoding strings using the sequence of even/odd transformations (i.e., parities) of Collatz iterations. Each character maps to a number, and its Collatz parity path forms a unique binary sequence from it's ASCII code.

This paper introduces the CPE algorithm, a deterministic mapping from characters to parity codes, along with a decoder capable of reconstructing the original string from a stream of parity bits *when using explicit delimiters*. We also explore prefix-free properties of these sequences and evaluate the theoretical potential of this encoding scheme.

2 The Journey to Collatz Parity Encoding

The development of the Collatz Parity Encoding (CPE) algorithm was a process of iterative insight, experimentation, and problem-solving.

2.1 Initial Idea: Character to Unary Encoding

The journey began with a straightforward concept: encoding each character in a string as a sequence of ones. This was done by taking the character's zero-based position in the alphabet, denoted as x, adding two, and then applying the Collatz iteration to that number. For instance, the letter A corresponds to the value 2 $(0 + 2)$, B to 3 $(1 + 2)$, and so on.

By doing this, I was able to transform plaintext characters into an unreadable sequence of ones, effectively using the Collatz conjecture as a unary encoder. This initial step revealed the potential to represent text through Collatz-based sequences, but decoding remained a challenge.

2.2 Encountering the Decoding Challenge

This decoding obstacle was a major stumbling block. Without explicit delimiters or length markers, the pure sequences of ones were indistinguishable when concatenated. The idea needed a breakthrough to allow for reliable and reversible decoding without relying on extra metadata.

2.3 Inspiration from Binary Parity Strings

The breakthrough emerged from revisiting the behavior of the Collatz iteration and representing each integer by the binary sequence of parity bits—indicating whether each step was even or odd—encountered until reaching 1. These parity sequences uniquely characterize each number's Collatz trajectory.

Initially, encoding characters by converting their alphabet positions into sequences of ones proved too ambiguous, as these unary codes were not self-delimiting; concatenated sequences could merge without clear boundaries, complicating decoding.

By shifting to parity-based encoding, each character's position was represented by its unique Collatz parity sequence, which inherently contained enough structure to allow reconstruction of the original number solely from the parity bits.

However, when concatenating multiple parity sequences to encode longer text, a decoding challenge remained: how to segment the combined string back into individual parity codes without ambiguity.

The key insight was identifying the delimiter sequence "11", which never appears within any valid Collatz parity sequence. By appending "11" after each character's parity code during encoding, the combined string became unambiguous and easily separable.

Decoding was then achieved by splitting the encoded string on the "11" delimiter, reversing each isolated parity code back to its corresponding number, and mapping back to the original character. This approach ensured a compact, fully reversible encoding without requiring additional metadata.

Listing 1: Section 6]Finding an unused delimiter in Collatz parity codes for printable ASCII characters. Collatz parity generation logic is introduced in Section 6

```
import string

printable_chars = string.printable.replace("\n", "").replace("\r", "
   ")
char_map = {char: idx for idx, char in enumerate(printable_chars,
   start=2)}

sequence = []

for c, idx in char_map.items():
    sequence.append(idx)

parity_array = []

for s in sequence:
    parity_array.append(generate_collatz_parity(s))

print(find_unused_flag(parity_array))
```

2.4 From Insight to Implementation

With binary parity strings as the core encoding units, the next step involved developing robust algorithms for generating parity sequences from character indices and decoding them back. An additional challenge was ensuring that parity codes could be deconstructed to allow for accurate individual character decoding without collisions.

The resulting Collatz Parity Encoding algorithm synthesizes mathematical insight with practical encoding needs, offering a novel, elegant approach to reversible string encoding inspired by the fascinating dynamics of the Collatz conjecture.

3 Background: The Collatz Conjecture

The Collatz function is defined as:

$$f(n) = \begin{cases} n/2, & \text{if } n \equiv 0 \pmod 2 \\ 3n+1, & \text{if } n \equiv 1 \pmod 2 \end{cases}$$

Starting from any positive integer n, repeatedly applying f eventually reaches 1, although this remains unproven. Each iteration yields a parity: 0 if even, 1 if odd. This forms a binary parity sequence for n, terminating at 1.

4 Encoding and Decoding with CPE

4.1 Encoding (CPE Encode)

- To generate a numerical sequence from a string, each character is mapped to its corresponding ASCII code

$$\text{Let } S = (s_1, s_2, \ldots, s_n) \text{ be a string of } n \text{ characters.}$$
$$\text{Define the ASCII mapping function } A(s_i) = \text{ord}(s_i)$$
$$\text{Then the ASCII sequence is: } (A(s_1), A(s_2), \ldots, A(s_n))$$

- Define the Collatz function:

$$f(n) = \begin{cases} \frac{n}{2} & \text{if } n \equiv 0 \pmod 2 \\ 3n+1 & \text{if } n \equiv 1 \pmod 2 \end{cases}$$

- For each n, generate the Collatz parity sequence $P(n) = (p_1, p_2, \ldots, p_k)$, where

$$p_i = \begin{cases} 0 & \text{if } n_i \text{ is even at step } i \\ 1 & \text{if } n_i \text{ is odd at step } i \end{cases}$$

  iterating f until $n_i = 1$.

- Concatenate parity sequences for all characters in the string $s = c_1 c_2 \ldots c_m$, inserting the delimiter `11` between each to produce the final encoded bitstream:

$$E(s) = P(\phi(c_1)) \parallel \texttt{11} \parallel P(\phi(c_2)) \parallel \texttt{11} \parallel \ldots \parallel \texttt{11} \parallel P(\phi(c_m))$$

4.2 Decoding (Using Delimiters)

- Let $D \subseteq \{0,1\}^*$ be the set of parity sequences generated by the Collatz Parity Encoding.

- The encoded bitstream $B = b_1 b_2 \ldots b_n$ consists of parity sequences concatenated with the delimiter `11` after each code:

$$B = P(\phi(c_1)) \parallel \texttt{11} \parallel P(\phi(c_2)) \parallel \texttt{11} \parallel \ldots \parallel P(\phi(c_m)) \parallel \texttt{11}$$

- To decode, split the bitstream B using the delimiter `11`, discarding any empty segments:

$$B \xrightarrow{\text{split on } \texttt{11}} \{p_1, p_2, \ldots, p_m\}, \quad \text{where each } p_i \in D \text{ and } p_i \neq \varepsilon$$

- Apply the reverse parity function $\psi(p_i)$ to each segment to recover the original integer:

$$\psi(p_i) = n_i \in \mathbb{N}$$

- Map each integer n_i back to its corresponding ASCII character using $c_i = \text{chr}(n_i)$

- Reconstruct the string:

$$s = c_1 c_2 \ldots c_m$$

- This method ensures accurate decoding as long as each parity code ends with the delimiter `11` and the set D is prefix-free.

5 Character Mapping and Encoding Logic

Each character in the input string is first mapped to it's ASCII code. This integer is then passed to the `generate_collatz_parity` function, which computes a binary sequence by applying the Collatz function and recording the parity (even = 0, odd = 1) of each step, excluding the final 1.

Because Collatz paths vary in length and structure even for consecutive integers, the resulting parity codes differ not only in content but also in bit length. For example, the ASCII characters `'A'` and `'B'`, despite being adjacent, may yield parity codes of very different lengths. This reflects the unpredictable, chaotic nature of Collatz trajectories.

To ensure the integrity of the encoding process, we verified that the mapping is both **injective** (no two characters produce the same parity sequence) and **deterministic** (the same input string always yields the same output). These properties make the encoding **collision-free** and **stable**, which are essential for reliable decoding.

While the variable-length codes may reduce compression efficiency, they do not affect correctness or reversibility. The inclusion of a fixed delimiter (`"11"`) after each parity code ensures accurate splitting during decoding, regardless of code length.

5.1 Alphabetical Indexing

Initially, CPE encoding utilized a mapping in which each letter was translated to its corresponding index in the English alphabet. However, problems arose when applying the Collatz Conjecture to these numbers. For example, if we assume zero-based indexing (i.e., A = 0, B = 1), the resulting values cannot be processed meaningfully through the conjecture, since 0 is invalid and 1 terminates immediately. A temporary fix was to increase each alphabetical index by two, such that A = 2, B = 3. This adjustment worked well in practice. Nevertheless, while effective, this method is redundant when we can instead map each character directly to its ASCII code and generate binary strings using the same Collatz process—without the need to artificially adjust values.

6 Sample Python Implementation

This section highlights essential Python snippets from the Collatz Parity Encoding (CPE) algorithm implementation.

Listing 2: Generating the Collatz parity sequence

```
import string

def generate_collatz_parity(n: int) -> str:
    parity = []
    while n != 1:
        parity.append("1" if n % 2 else "0")
        if n % 2 == 0:
            n //= 2
        else:
            n = 3 * n + 1
    return "".join(parity)
```

This function generates the parity sequence for a given integer n, encoding each Collatz step as `'0'` for even and `'1'` for odd, excluding the final step reaching 1.

Listing 3: Encoding a string into concatenated parity codes

```
def encode(plain_text: str) -> str:
    sequence = generate_sequence(plain_text)
    parity_list = [generate_collatz_parity(num) for num in sequence]
    # Use "11" as a delimiter between parity codes for metadata-free parsing
    encoded = [parity + "11" for parity in parity_list]
    return "".join(encoded)
```

Here, the plaintext string is first mapped to a sequence of integer codes (based on it's ASCII code), then converted into parity sequences. The delimiter `"11"` marks boundaries between encoded characters to enable metadata-free parsing.

Listing 4: Decoding concatenated parity codes back to text

```
def decode(parity_str: str) -> str:
    parity_arr = [p for p in parity_str.split("11") if p]
    sequence = []
    for p in parity_arr:
        n = reverse_from_parity(p)
        if n is None:
            raise ValueError(f"Invalid parity sequence: {p}")
        sequence.append(n)
    char_array = reverse_sequence(sequence)
    return "".join(char_array)
```

The decoder splits the encoded string at each `"11"` delimiter, reverses each resulting parity code to recover the original integer, and then maps these integers back to their corresponding ASCII characters.

6.1 Sample Parity Codes

- 'a' → parity: 01010001010010001000011
- 'b' → parity: 10101001010000101000101001000100011

7 Theoretical Insights

7.1 Formal Definition of Self-Delimiting Codes

A code is said to be *self-delimiting* (or *prefix-free*) if no codeword is a prefix of any other codeword. Formally, for any two distinct codewords x and y, neither x is a prefix of y nor y is a prefix of x. This property guarantees that a concatenated sequence of codewords can be uniquely and unambiguously parsed in a left-to-right manner without requiring explicit delimiters or length information.

In our experiments with the ASCII character set defined, we verified that the generated codewords do not satisfy prefix-freeness. This failure necessitates the use of explicit delimiters as described in the preceding section.

7.2 Towards a Prefix-Free Encoding

In the current implementation of the CPE algorithm, each character is encoded as a binary parity sequence representing its Collatz trajectory, with the delimiter "11" appended after each code to ensure unambiguous decoding.

However, these raw parity sequences are **not prefix-free**—some parity codes are strict prefixes of others—which means decoding concatenated sequences without delimiters would lead to ambiguity and errors.

To overcome this limitation and enable delimiter-free decoding, several approaches could be explored:

- **Code Extension:** Modify or extend parity sequences by appending unique suffix patterns to ensure no code is a prefix of another. This could be done by adding carefully chosen bits that preserve reversibility while guaranteeing prefix-freeness.
- **Variable-Length Encoding with Prefix Codes:** Integrate classic prefix-free coding techniques such as Huffman or Elias codes on top of parity sequences, effectively combining Collatz-based encoding with established prefix-free schemes.
- **Self-Delimiting Codes:** Design a self-delimiting scheme where each parity sequence encodes its own length implicitly, for example by embedding length information or unique termination patterns within the code, ensuring no code can be confused as a prefix of another.

- **Delimiter-Based Hybrid:** Continue using a delimiter but reduce its length or frequency by grouping multiple characters into blocks, then encoding each block with parity sequences separated by a minimal delimiter or marker, reducing overhead while retaining reliable decoding.

Exploring these ideas could lead to a fully self-delimiting, prefix-free version of the CPE algorithm, eliminating the need for explicit delimiters like "11" and enabling more efficient, streaming-friendly encoding and decoding.

Future work will focus on evaluating these approaches and formally proving prefix-free properties, while balancing encoding complexity and decoding efficiency.

8 Related Work

The design of efficient and reversible data encoding schemes has a rich history, particularly in the domain of self-delimiting and prefix-free codes. These codes enable unambiguous decoding of concatenated codewords without requiring explicit length metadata or delimiters, making them foundational in data compression and information theory.

Traditional self-delimiting codes such as Elias gamma, delta, and omega codes [1] provide variable-length representations of integers with strong prefix-free properties. These codes leverage unary and binary numeral systems to achieve compactness and efficient decoding. For instance, Elias gamma codes represent positive integers with a unary length prefix followed by a binary suffix, allowing for deterministic parsing in a stream of concatenated codewords.

Similarly, Huffman coding [2] is a well-known method for constructing optimal prefix codes based on symbol frequencies. Huffman codes minimize the expected code length and are widely used in practical compression algorithms. However, they require knowledge of symbol probabilities and a codebook for encoding and decoding, which introduces overhead when metadata transmission is restricted.

More recent approaches have explored combinatorial and mathematical constructs for encoding, including those based on continued fractions, arithmetic coding, and context modeling. Nonetheless, these methods typically rely on probabilistic models or external metadata to maintain reversibility and efficiency.

The Collatz Parity Encoding (CPE) algorithm introduced in this work distinguishes itself by leveraging the unique parity sequences generated through the iterative Collatz function. Unlike classical schemes, CPE encodes characters as parity patterns intrinsic to the chaotic trajectories of integer sequences under the Collatz map. This mathematical grounding provides a novel, deterministic mapping from characters to variable-length parity codes that are empirically shown to be prefix-free.

While CPE currently employs an explicit delimiter for robust decoding, its underlying prefix-free parity structure suggests potential for fully metadata-free decoding in future iterations. This contrasts with traditional codes, which depend heavily on designed prefix properties or additional metadata for unambiguous parsing.

To the best of our knowledge, the use of the Collatz conjecture's parity sequences as a basis for reversible encoding has not been explored in prior work. Thus, CPE contributes a

new paradigm at the intersection of number theory and information encoding, with promising implications for compact, self-delimiting data representation.

9 Future Work

Future developments of the Collatz Parity Encoding (CPE) algorithm will focus on several key areas:

- Formally proving prefix-freeness or establishing conditions for self-delimiting behavior across arbitrary character sets, enabling delimiter-free and streaming decoding.
- Implementing self-delimiting decoding algorithms that remove the need for explicit delimiters such as `"11"`, improving compression efficiency and simplifying parsing.
- Enhancing decoding performance through data structures like compressed tries or automata to optimize search and matching of parity sequences.
- Conducting thorough analyses of entropy, compression ratios, and comparative benchmarks against established encoding schemes.
- Extending the encoding framework to support multibyte and variable-length character encodings such as UTF-8, broadening applicability to international text.
- Investigating practical applications of CPE in secure communication, steganography, or novel data compression tools that leverage the number-theoretic properties of Collatz sequences.
- Exploring integration with streaming protocols and real-time data processing pipelines where delimiter-free encoding could reduce overhead.

10 Conclusion

The CPE algorithm introduces a novel reversible encoding scheme based on Collatz parity dynamics, offering a unique approach to data representation. While the generated parity codes exhibit promising prefix-free properties within tested ranges, the current implementation relies on an explicit delimiter sequence ("11") to separate codes during decoding. This means that full self-delimiting behavior has not yet been achieved. Future work aims to explore methods for removing this dependency to enable delimiter-free, streaming decoding, which could enhance compression efficiency and practical applicability.

References

[1] P. Elias, *Universal codeword sets and representations of the integers*, IEEE Transactions on Information Theory, vol. 21, no. 2, pp. 194–203, 1975.

[2] D. A. Huffman, *A Method for the Construction of Minimum-Redundancy Codes*, Proceedings of the IRE, vol. 40, no. 9, pp. 1098–1101, 1952.

[3] J. C. Lagarias, *The 3x+1 Problem: An Annotated Bibliography (1963–1999)*, Mathematics of Computation, vol. 69, no. 231, pp. 755–768, 2000.